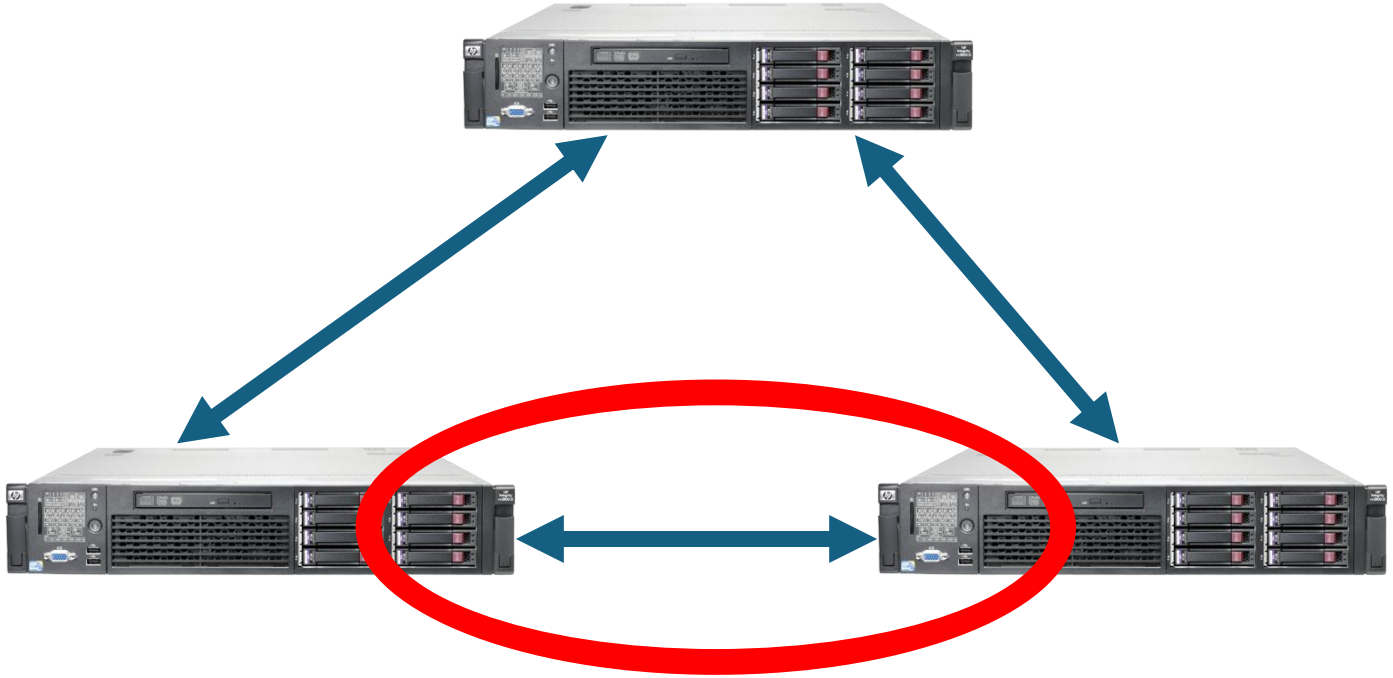
# CSE469 Lecture 2

## Network Communication and Remote Procedure Calls

The following slides are based on:

- An edited version of slides by professor Wyatt Lloyd taught in Princeton's COS418 course (CC license).
- An edited version of slides provided by textbook authors (M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017)
- Code examples from different sources (cited within)

# Distributed Systems, What?



1) Multiple computers
2) Connected by a network
3) Doing something together

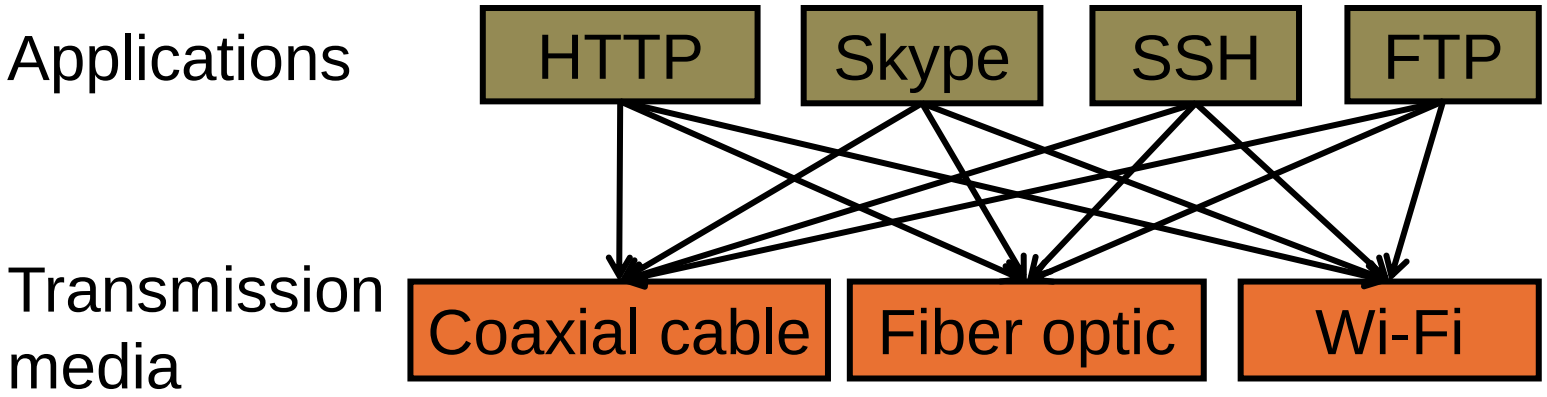
# Outline

- Review of Networking Concepts
- Remote Procedure  Calls
- RPCs in Go
- Handling Failure in RPCs

# The problem of communication

- Process on Host A wants to talk to process on Host B
  - A and B must agree on the meaning of the bits being sent and received at many different levels, including:
    - How many volts is a 0 bit, a 1 bit?
    - How does receiver know which is the last bit?
    - How many bits long is a number?

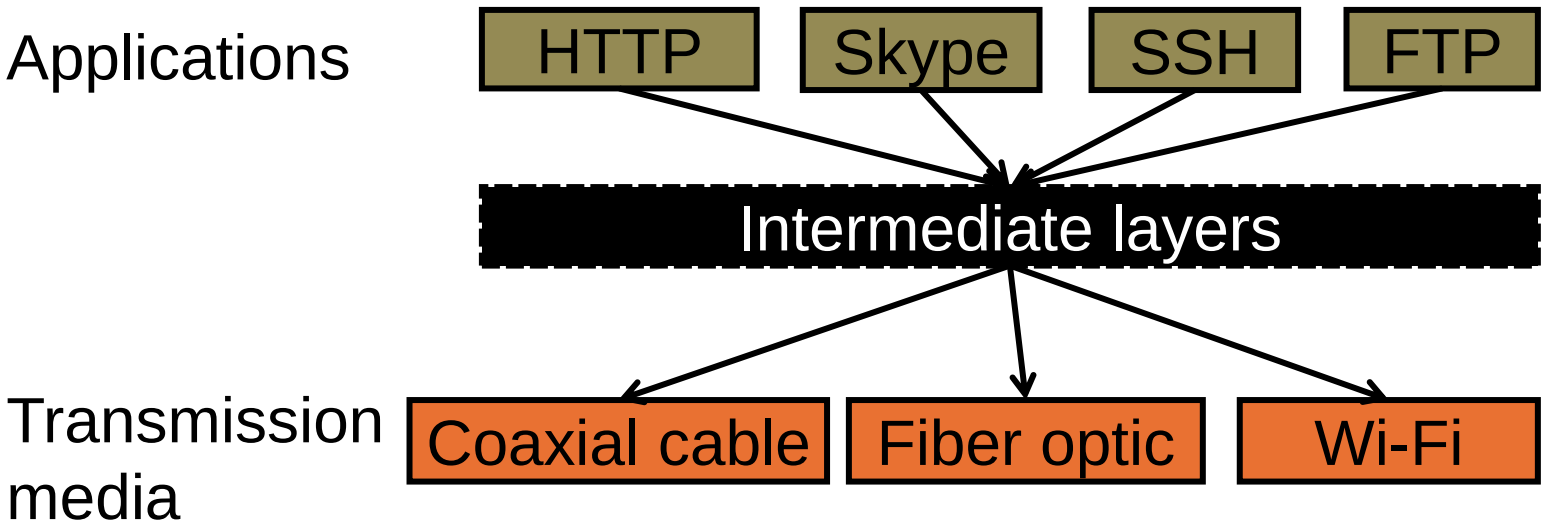
# The problem of communication




- Re-implement every application for every new underlying transmission medium?
- Change every application on any change to an underlying transmission medium?

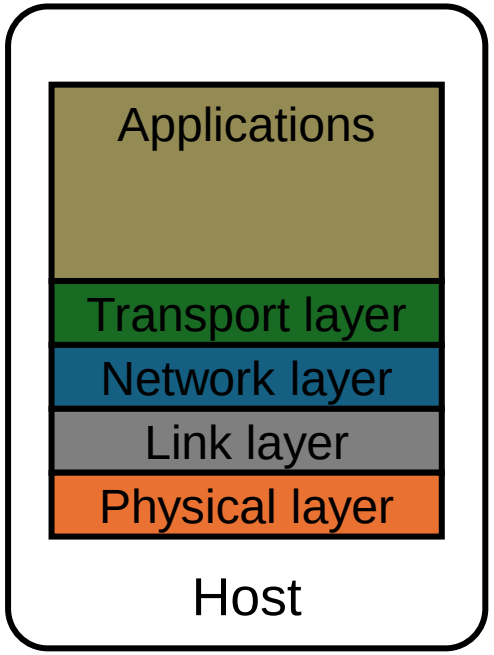
- No! But how does the Internet design avoid this?

# Solution: Layering




- Intermediate **layers** provide a set of abstractions for applications and media
- New applications or media need only implement for intermediate layer's interface

# Layering in the Internet

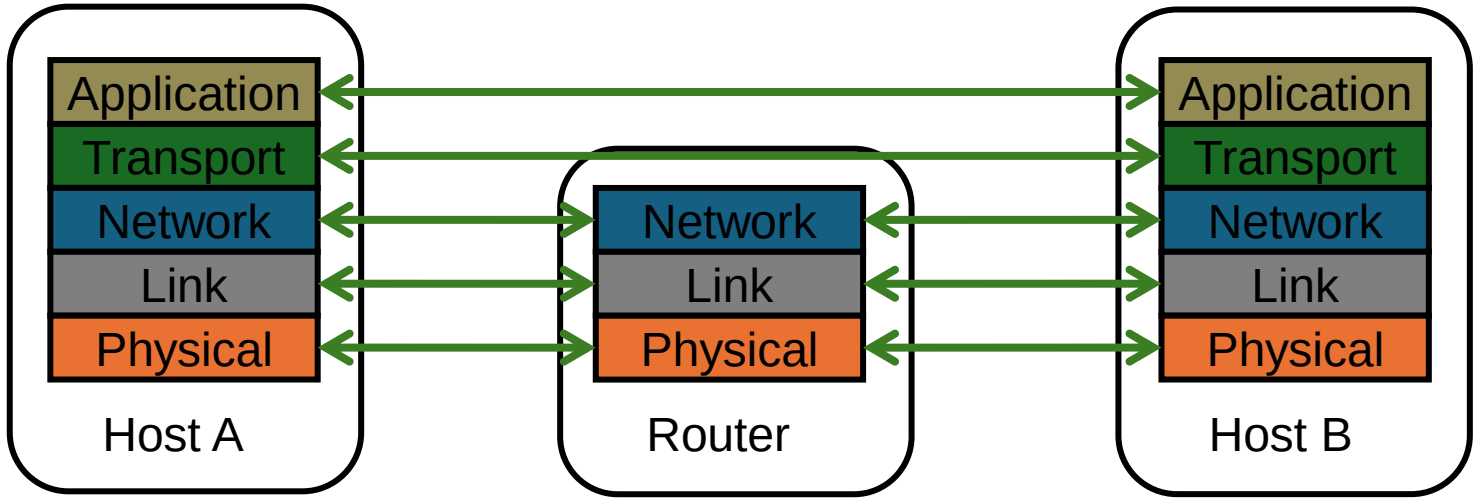



- **Transport:** Provide end-to-end communication between processes on different hosts
- **Network:** Deliver packets to destinations on other (heterogeneous) networks
- **Link:** *Moving data from one node to an adjacent node over a single communication link*
- **Physical:** Moves bits between two hosts connected by a physical link

# Logical communication between layers

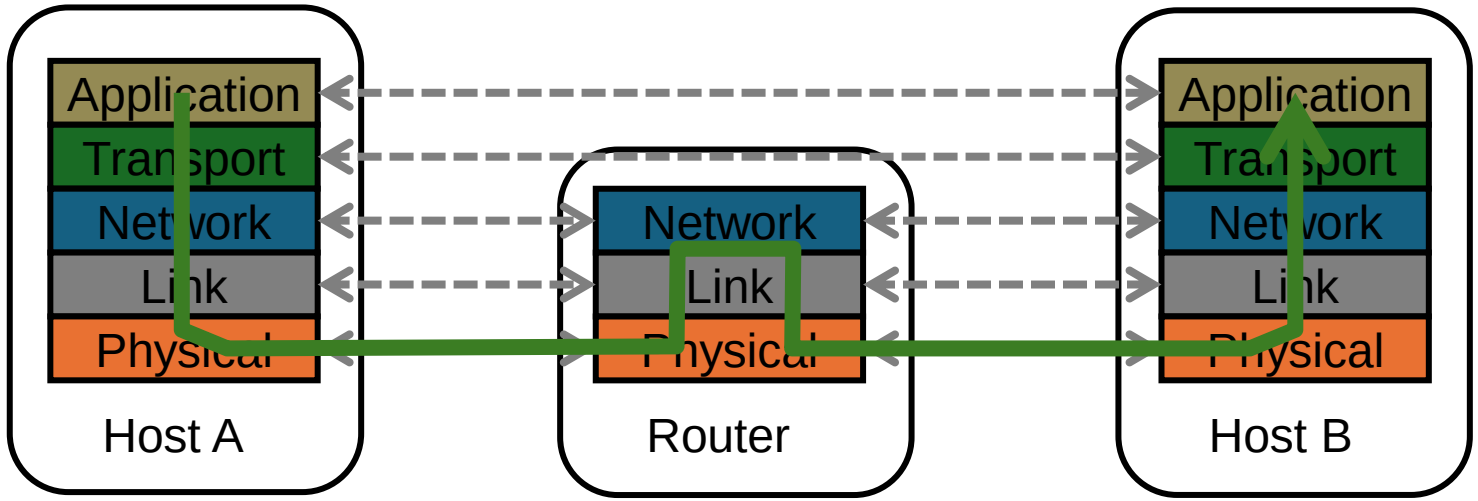
- How to forge agreement on the meaning of the bits exchanged between two hosts?

- **Protocol:** Rules that govern the format, contents, and meaning of messages
  - Each layer on a host interacts with its peer host's corresponding layer via the **protocol interface**



# Physical communication

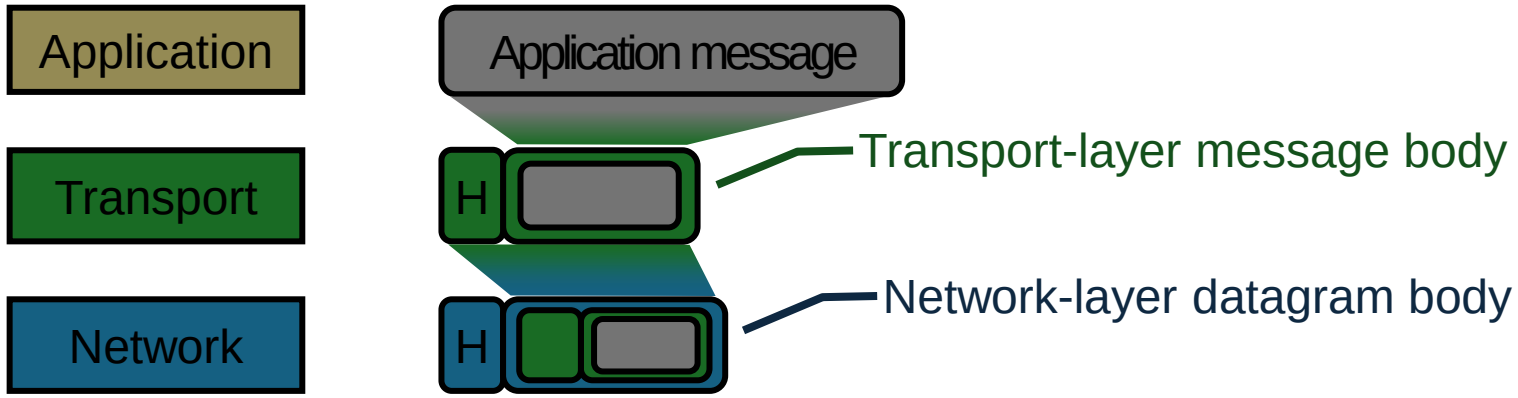
- Communication goes down to the **physical network**
- Then from **network** peer to peer
- Then up to the **relevant application**



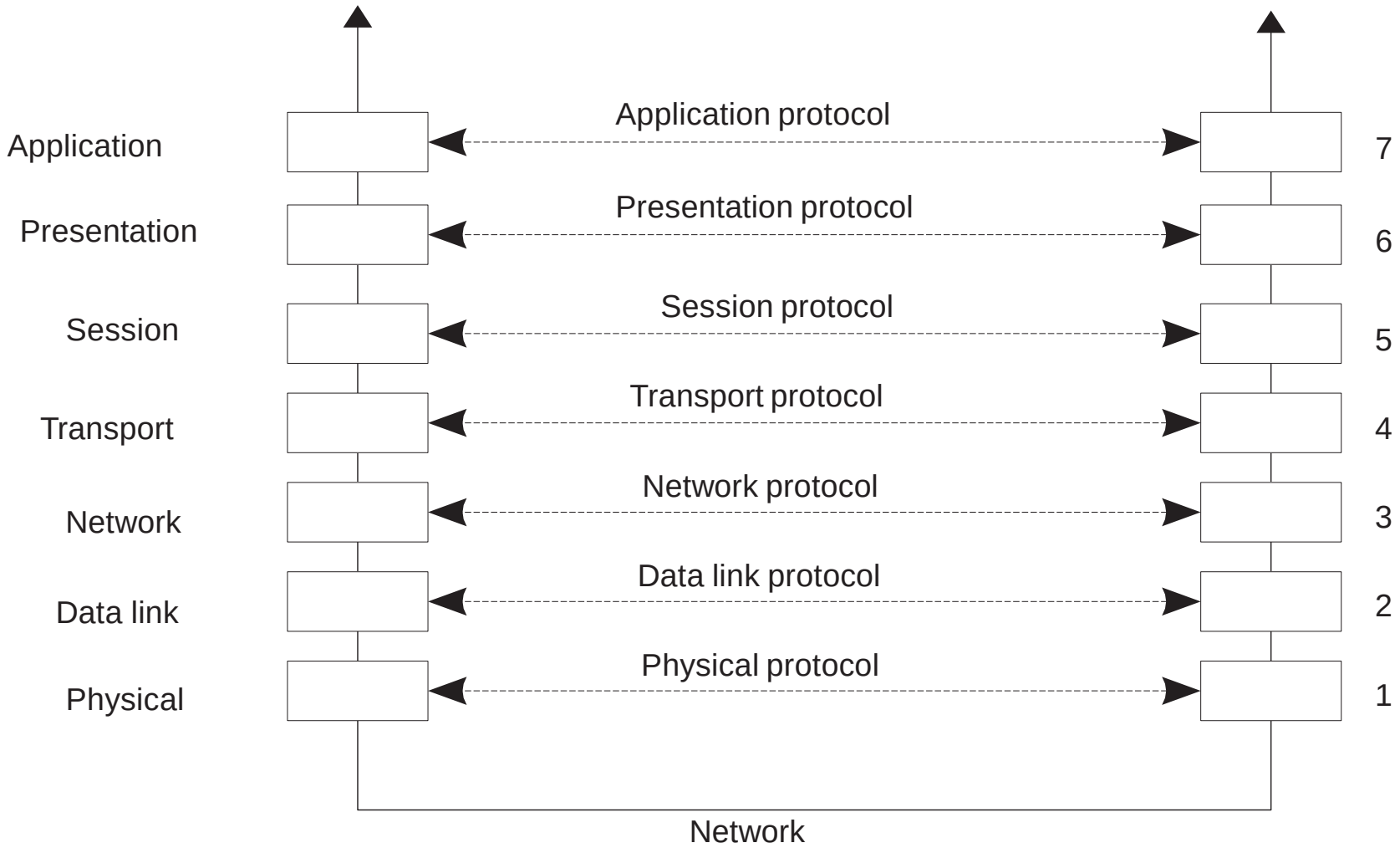
# Communication between peers

- How do peer protocols coordinate with each other?

- Layer attaches its own header (H) to communicate with peer
  - Higher layers' headers, data encapsulated inside message
    - Lower layers don't generally inspect higher layers' headers



# The OSI Reference Model



# Notes

- Due to the absence of shared memory, all communication in distributed systems is based on sending and receiving (low level) messages.
  - For many distributed systems, the lowest-level interface is that of the network layer.
  - Which property is violated when accessing remote objects is done by explicit network communication?

- In the next slide, we will split the application-layer protocols into two layers (middleware layer + application layer).
  - As we will see shortly, this will make things easier for the application-developers and improves transparency to some extent.

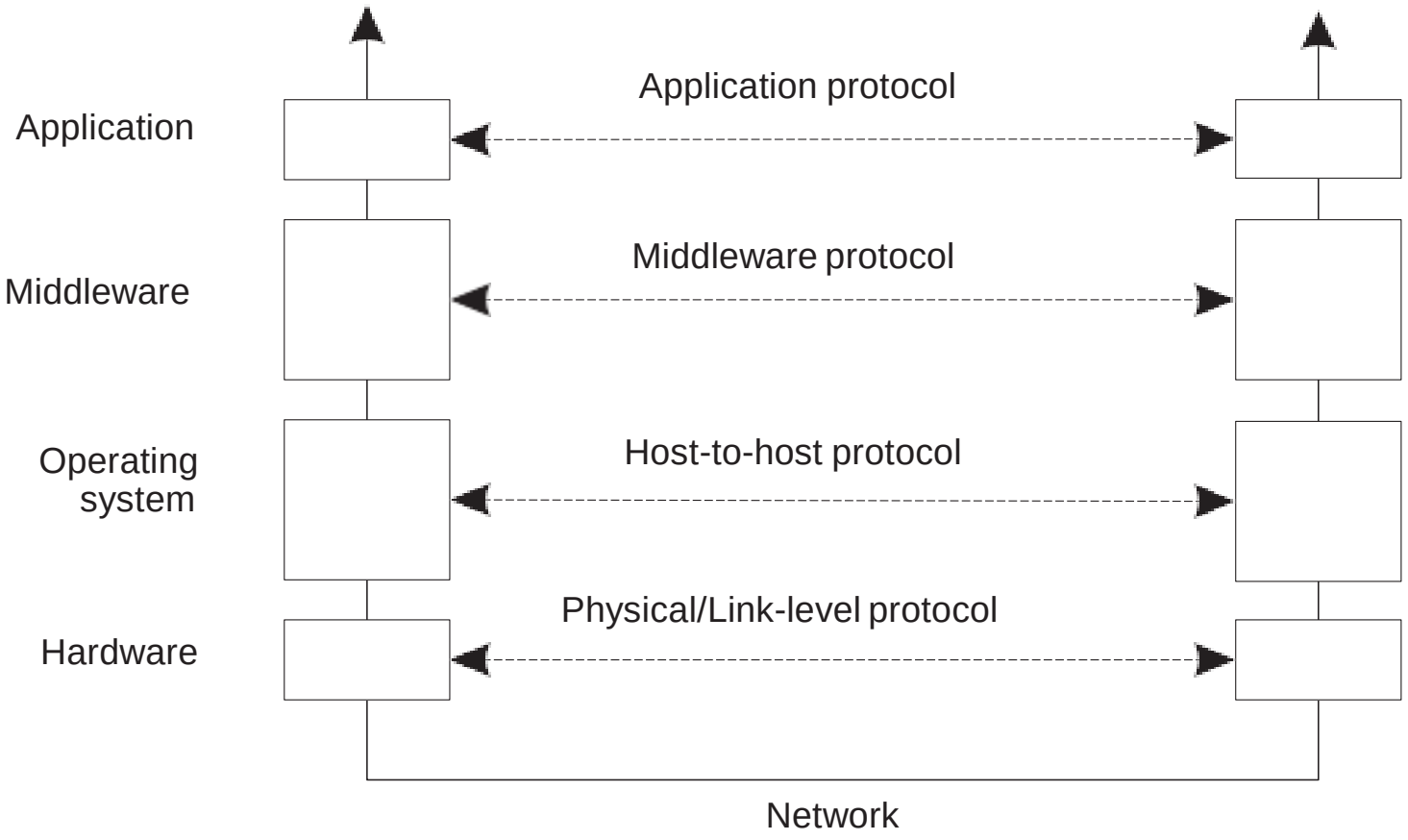
# Middleware layer

## Observation

Middleware is invented to provide common services and protocols that can be used by many different applications

- A rich set of communication protocols
- (Un)marshaling of data, necessary for integrated systems
- Naming protocols, to allow easy sharing of resources
- Security protocols for secure communication
- Scaling mechanisms, such as for replication and caching

# An adapted layering scheme



# Types of communication

Distinguish...

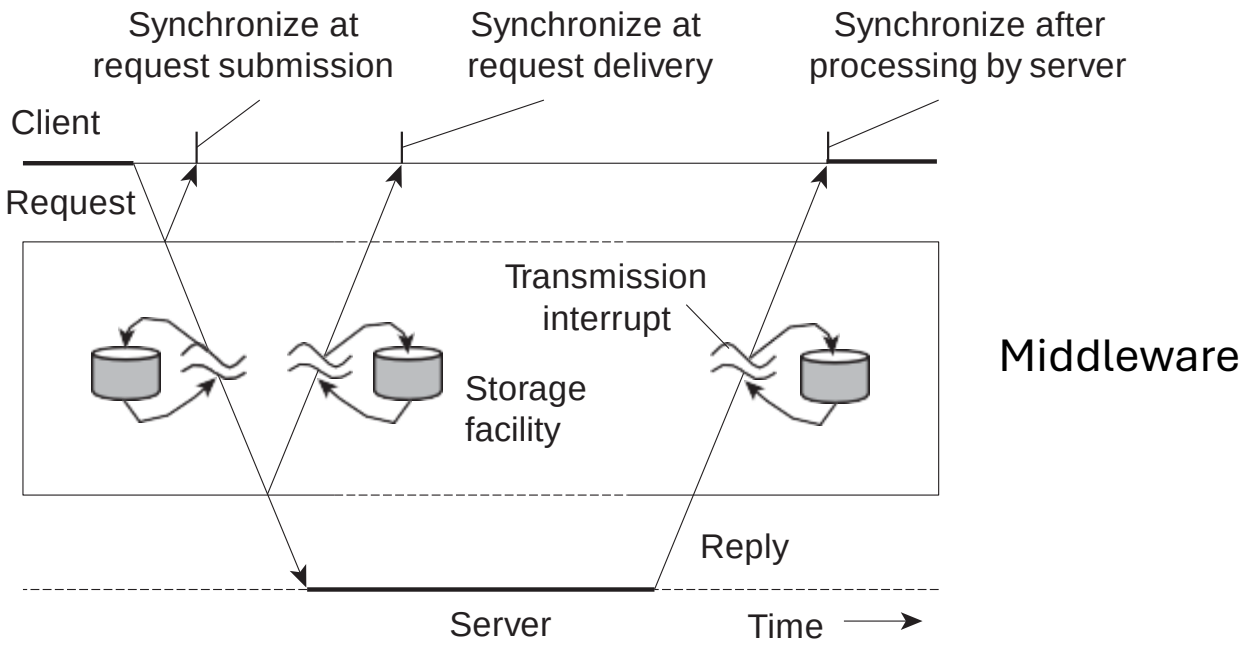
- Transient versus persistent communication
- Asynchronous versus synchronous communication

- Transient communication: The communication middleware discards message when it cannot be delivered at the next server, or at the receiver.
- Persistent communication: A message is stored at the communication middleware as long as it takes to deliver it.

- Asynchronous communication: The sender proceeds with execution immediately after sending the message.
- Synchronous communication: The sender waits before proceeding with the execution. There are different points where synchronization may take place.

# Types of communication

## Places for synchronization




- At request submission
- At request delivery
- After request processing

# Next

We will discuss two communication variants:

- **Remote procedure calls** (RPCs)[1]
  - Transient
  - Synchronous or asynchronous
- **Message-oriented communication [Next lecture]**
  - We will discuss transient and persistent variants.

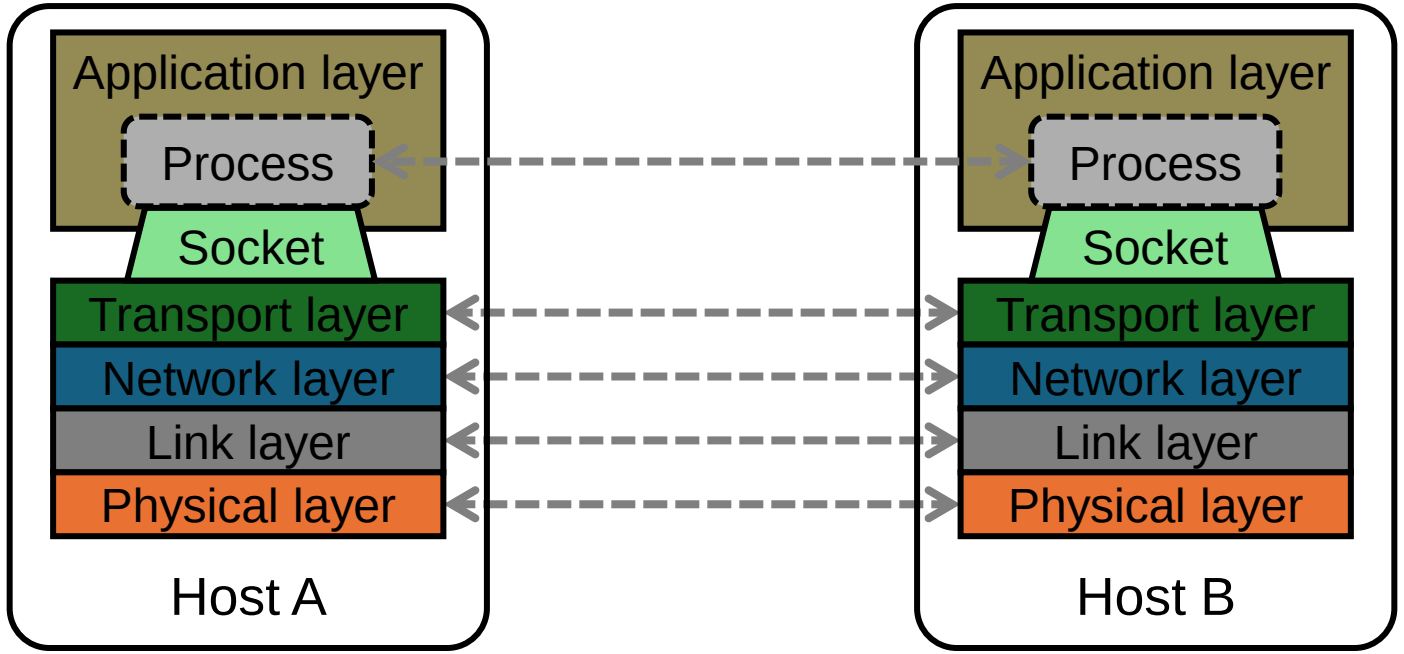
[1] We will consider the simplest form of RPCs (the transient variant only). There might be ways to make RPCs persistent by combining it with a technique from persistent communication methods.

# Remote Procedure Calls (RPCs)

# Network socket-based communication

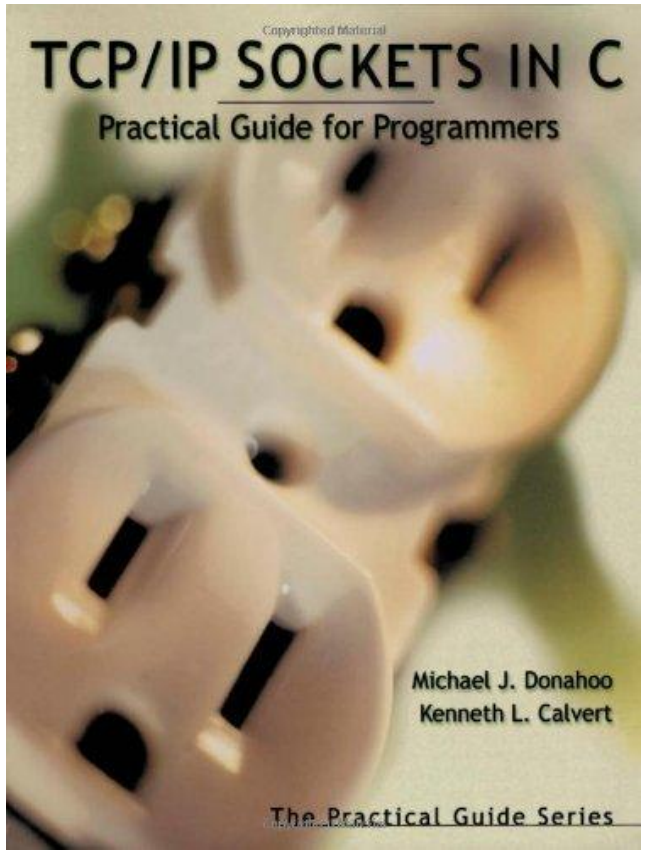
- Socket: The interface the OS provides to the network
  - Provides inter-process explicit message exchange

- Can build distributed systems atop sockets: send(), recv()
  - e.g.: `put(key,value)` → message



# Socket Programming in C Examples

- We will go through:
  - TCP Echo Server
  - TCP Echo Client



# Server Code (1/2)

```
echoServPort = atoi(argv[1]);  /* First arg:  local port */

/* Create socket for incoming connections */
if ((servSock = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP)) < 0)
    DieWithError("socket() failed");

/* Construct local address structure */
memset(&echoServAddr, 0, sizeof(echoServAddr));   /* Zero out structure */
echoServAddr.sin_family = AF_INET;                /* Internet address family */
echoServAddr.sin_addr.s_addr = htonl(INADDR_ANY); /* Any incoming interface */
echoServAddr.sin_port = htons(echoServPort);      /* Local port */

/* Bind to the local address */
if (bind(servSock, (struct sockaddr *) &echoServAddr, sizeof(echoServAddr)) < 0)
    DieWithError("bind() failed");

/* Mark the socket so it will listen for incoming connections */
if (listen(servSock, MAXPENDING) < 0)
    DieWithError("listen() failed");
```

# Server Code (2/2)

```
for (;;) /* Run forever */
{
    /* Set the size of the in-out parameter */
    clntLen = sizeof(echoClntAddr);

    /* Wait for a client to connect */
    if ((clntSock = accept(servSock, (struct sockaddr *) &echoClntAddr,
                           &clntLen)) < 0)
        DieWithError("accept() failed");

    /* clntSock is connected to a client! */

    printf("Handling client %s\n", inet_ntoa(echoClntAddr.sin_addr));

    HandleTCPClient(clntSock);
}
```

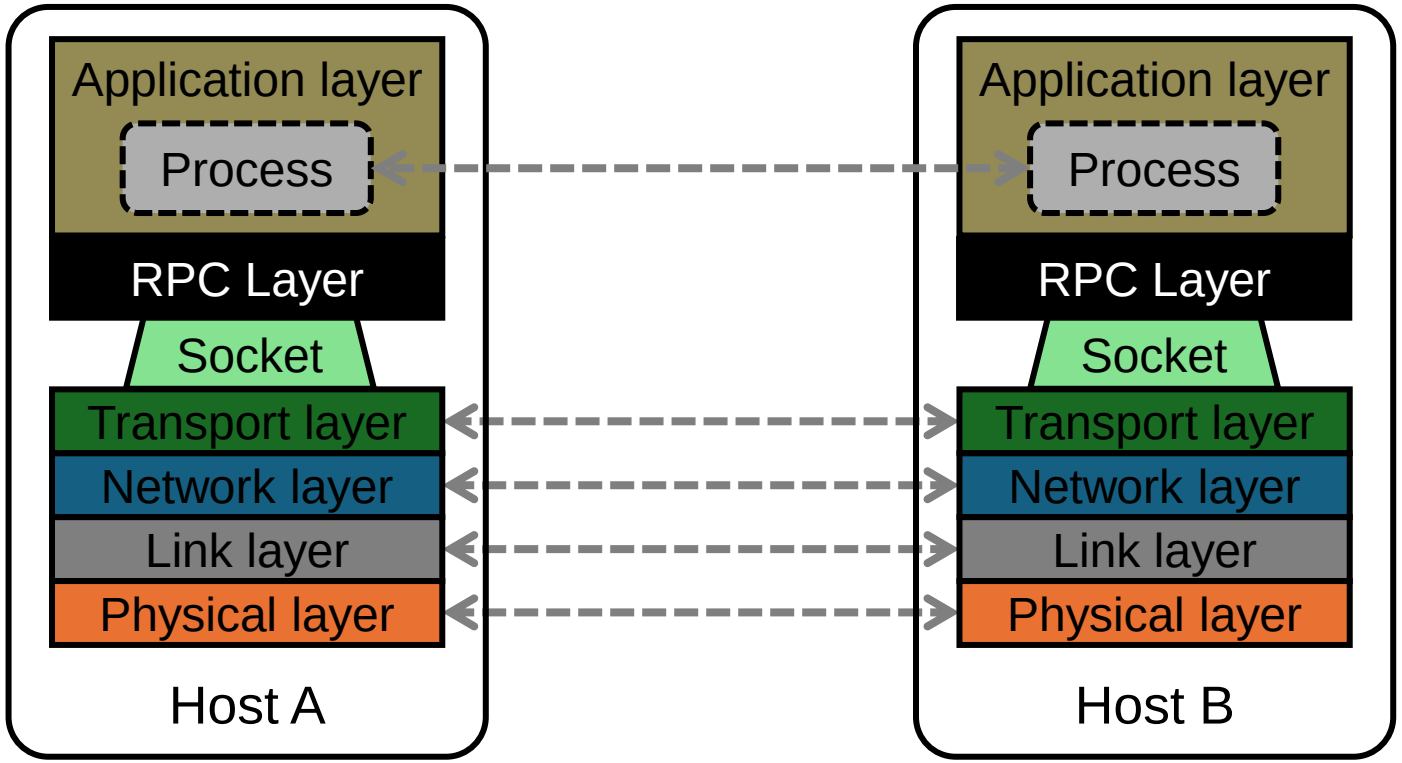
# Client Code

```
/* Create a reliable, stream socket using TCP */
if ((sock = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP)) < 0)
    DieWithError("socket() failed");

/* Construct the server address structure */
memset(&echoServAddr, 0, sizeof(echoServAddr));     /* Zero out structure */
echoServAddr.sin_family      = AF_INET;             /* Internet address family */
echoServAddr.sin_addr.s_addr = inet_addr(servIP);   /* Server IP address */
echoServAddr.sin_port        = htons(echoServPort); /* Server port */

/* Establish the connection to the echo server */
if (connect(sock, (struct sockaddr *) &echoServAddr, sizeof(echoServAddr)) < 0)
    DieWithError("connect() failed");

echoStringLen = strlen(echoString);          /* Determine input length */

/* Send the string to the server */
if (send(sock, echoString, echoStringLen, 0) != echoStringLen)
    DieWithError("send() sent a different number of bytes than expected");

/* Receive the same string back from the server */
totalBytesRcvd = 0;
printf("Received: ");                 /* Setup to print the echoed string */
while (totalBytesRcvd < echoStringLen)
{
    /* Receive up to the buffer size (minus 1 to leave space for
       a null terminator) bytes from the sender */
    if ((bytesRcvd = recv(sock, echoBuffer, RCVBUFSIZE - 1, 0)) <= 0)
        DieWithError("recv() failed or connection closed prematurely");
    totalBytesRcvd += bytesRcvd;   /* Keep tally of total bytes */
    echoBuffer[bytesRcvd] = '\0';  /* Terminate the string! */
    printf("%s", echoBuffer);      /* Print the echo buffer */
}
```

# Socket programming: still not great

- Lots for the programmer to deal with every time
  - How to separate different requests on the same connection?
  - How to write bytes to the network / read bytes from the network?
    - What if Host A's process is written in Go and Host B's process is in C++?
  - What to do with those bytes?
- Still pretty painful… have to worry a lot about the network

# Solution: Another layer!



# Why RPC?

- The typical programmer is trained to write single-threaded code that runs in one place

- Goal: Easy-to-program network communication that makes client-server communication transparent
  - Retains the “feel” of writing centralized code
    - Programmer needn’t think about the network

# RPCs are used Widely

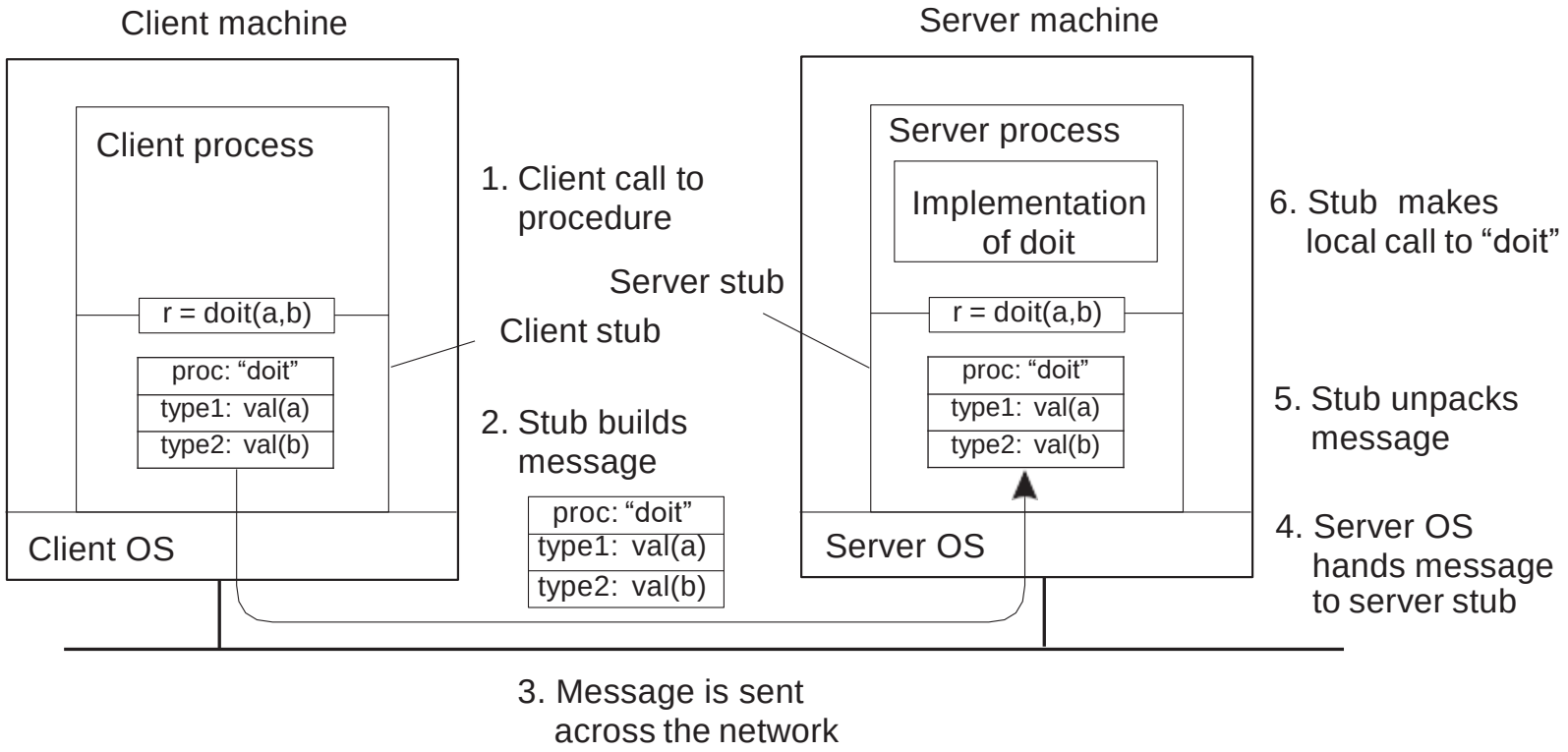
- Google gRPC: https://grpc.io/
- Facebook/Apache Thrift: https://github.com/facebook/fbthrift
- Twitter Finagle: https://twitter.github.io/finagle/
- …

# What's the goal of RPC?

- Within a single program, running in a single process, recall the well-known notion of a **procedure call**:
  - **Caller** pushes arguments onto stack,
    - jumps to address of **callee** function
  - **Callee** reads arguments from stack,
    - executes, puts return value in register,
    - returns to next instruction in caller

**RPC's Goal**: make communication appear like a local procedure call: way less painful than sockets…

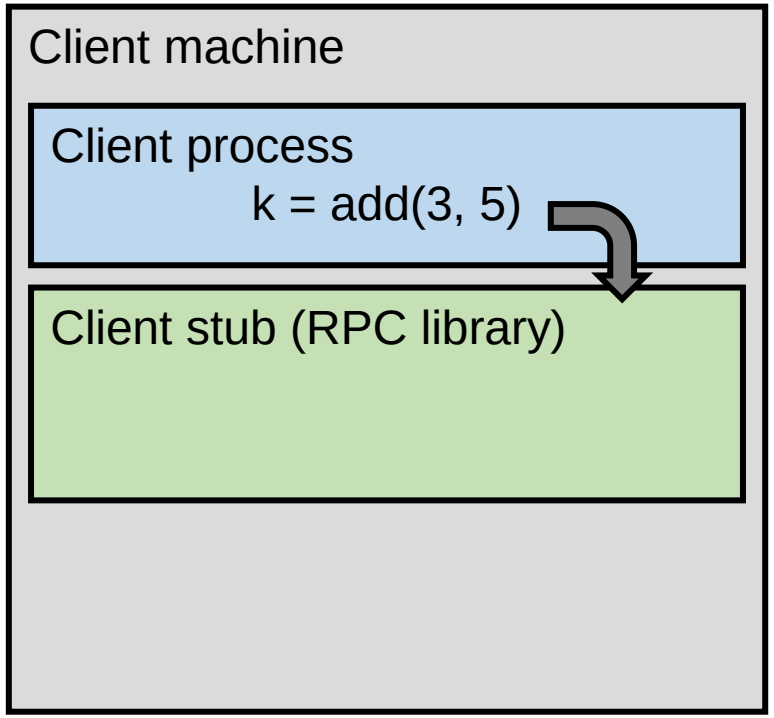
# Basic RPC operation




1. Client procedure calls client stub.
2. Stub builds message; calls local OS.
3. OS sends message to remote OS.
4. Remote OS gives message to stub.
5. Stub unpacks message
6. Server does local call; returns result to stub.
7. Stub builds message; calls OS.
8. OS sends message to client’s OS.
9. Client’s OS gives message to stub.
10. Client stub unpacks result; returns to client

# A day in the life of an RPC

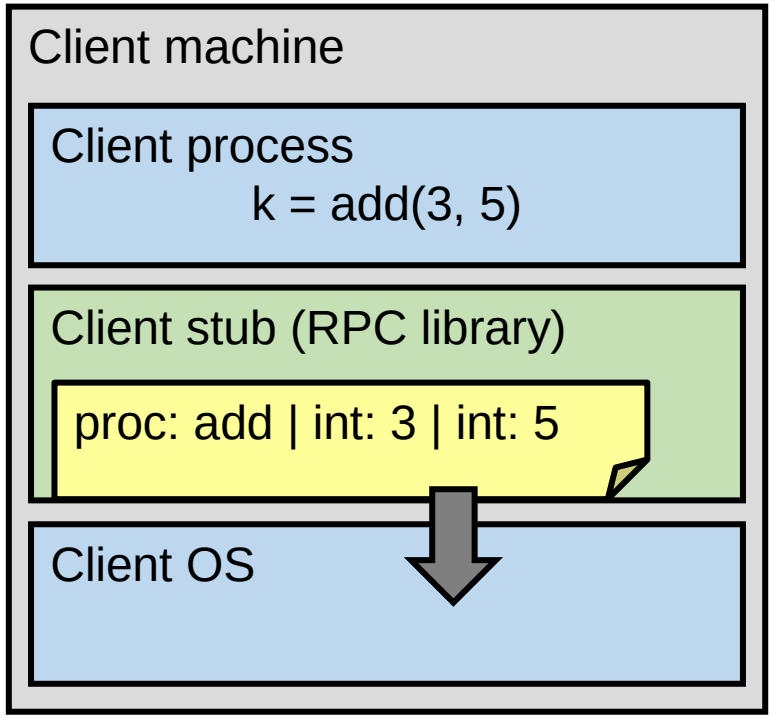
1. Client calls stub function (pushes parameters onto stack)



# A day in the life of an RPC

1. Client calls stub function (pushes parameters onto stack)

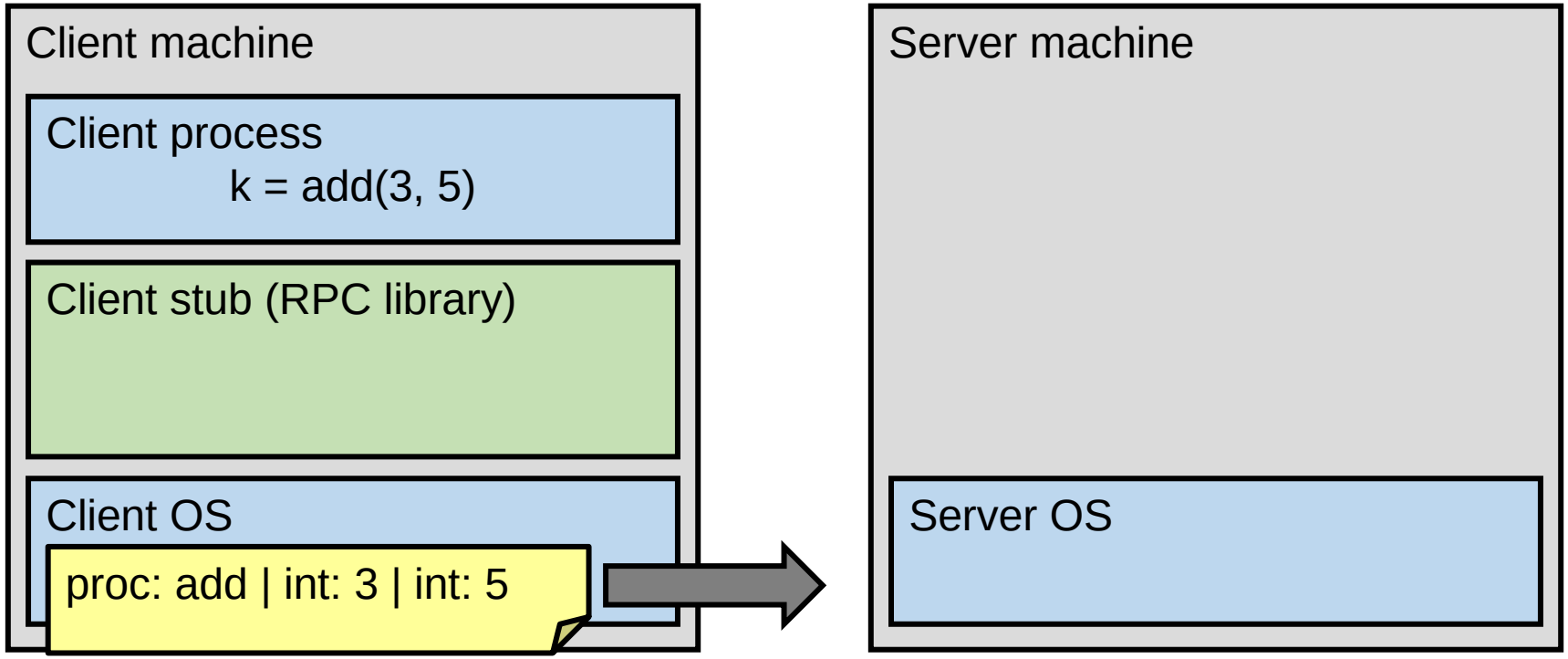
2. Stub marshals parameters to a network message



# A day in the life of an RPC

2. Stub marshals parameters to a network message

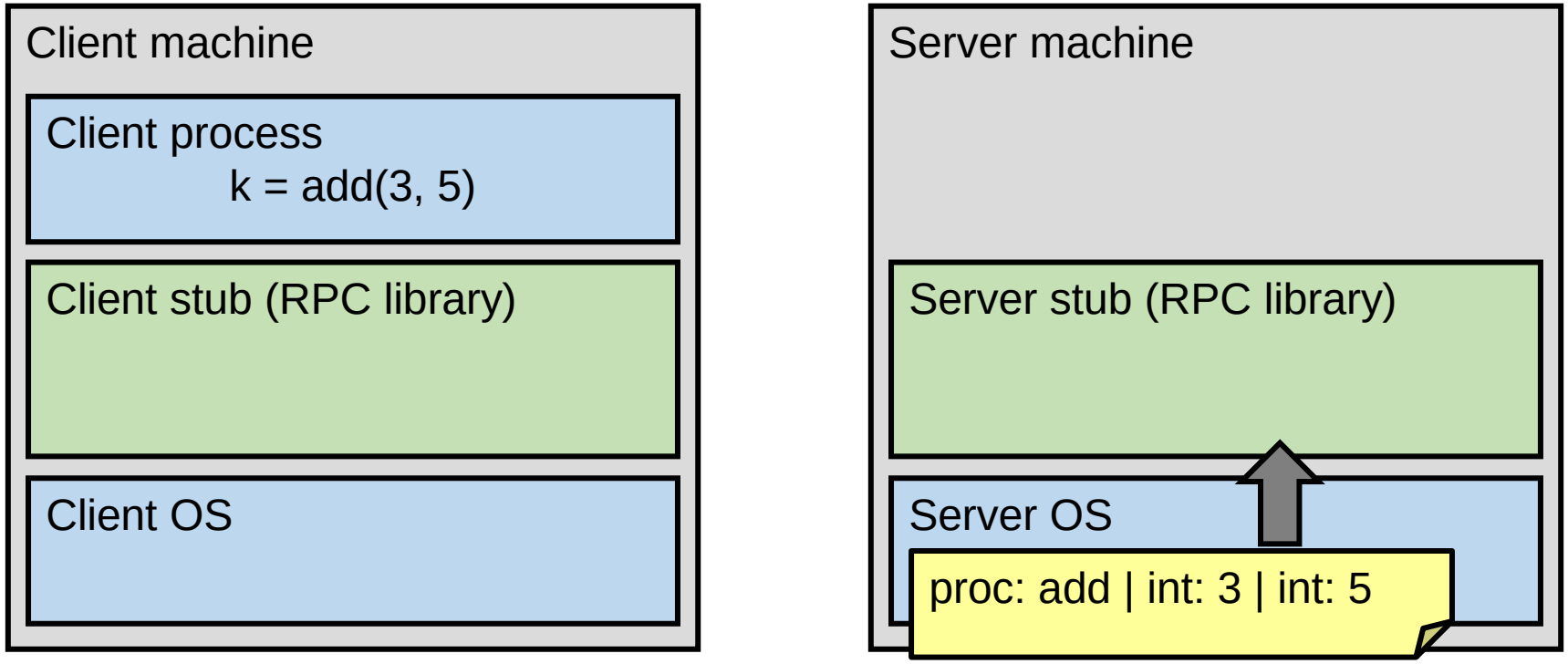
3. OS sends a network message to the server



# A day in the life of an RPC

3. OS sends a network message to the server

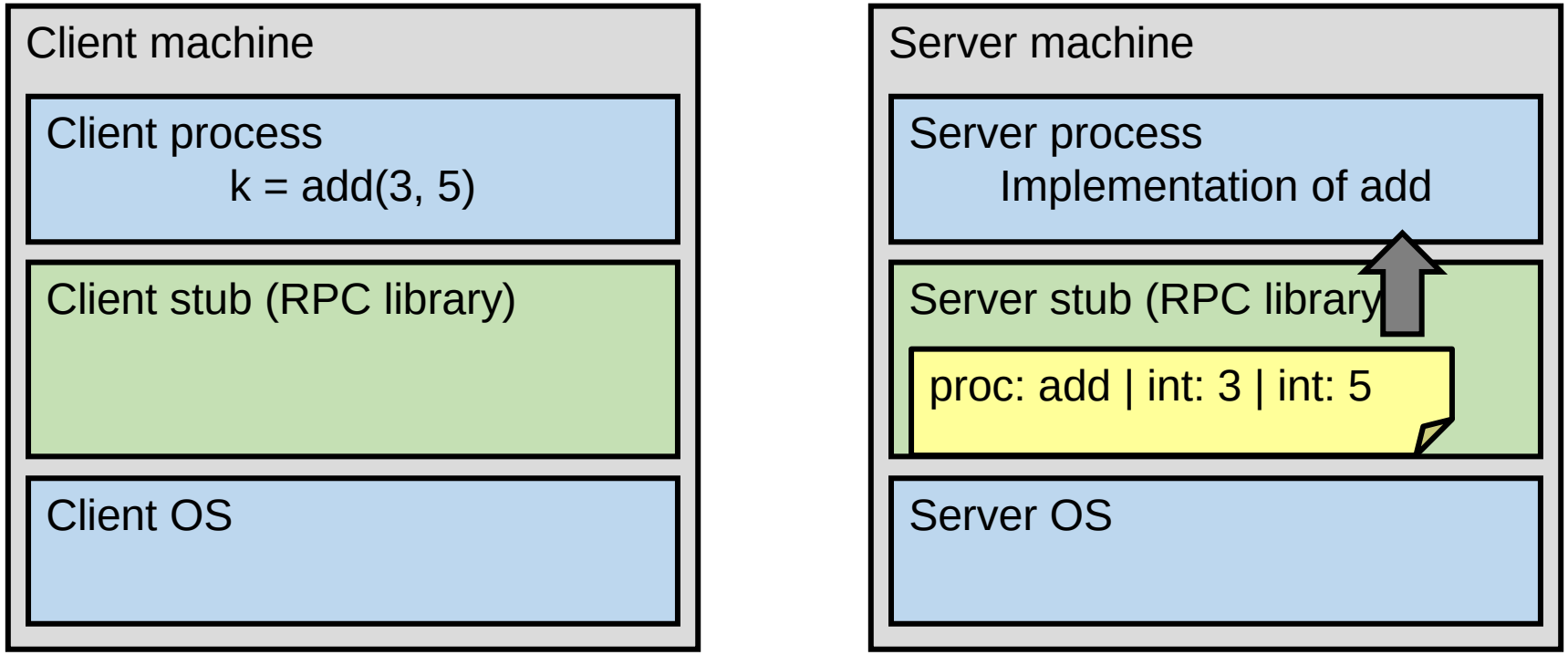
4. Server OS receives message, sends it up to stub



# A day in the life of an RPC

4. Server OS receives message, sends it up to stub

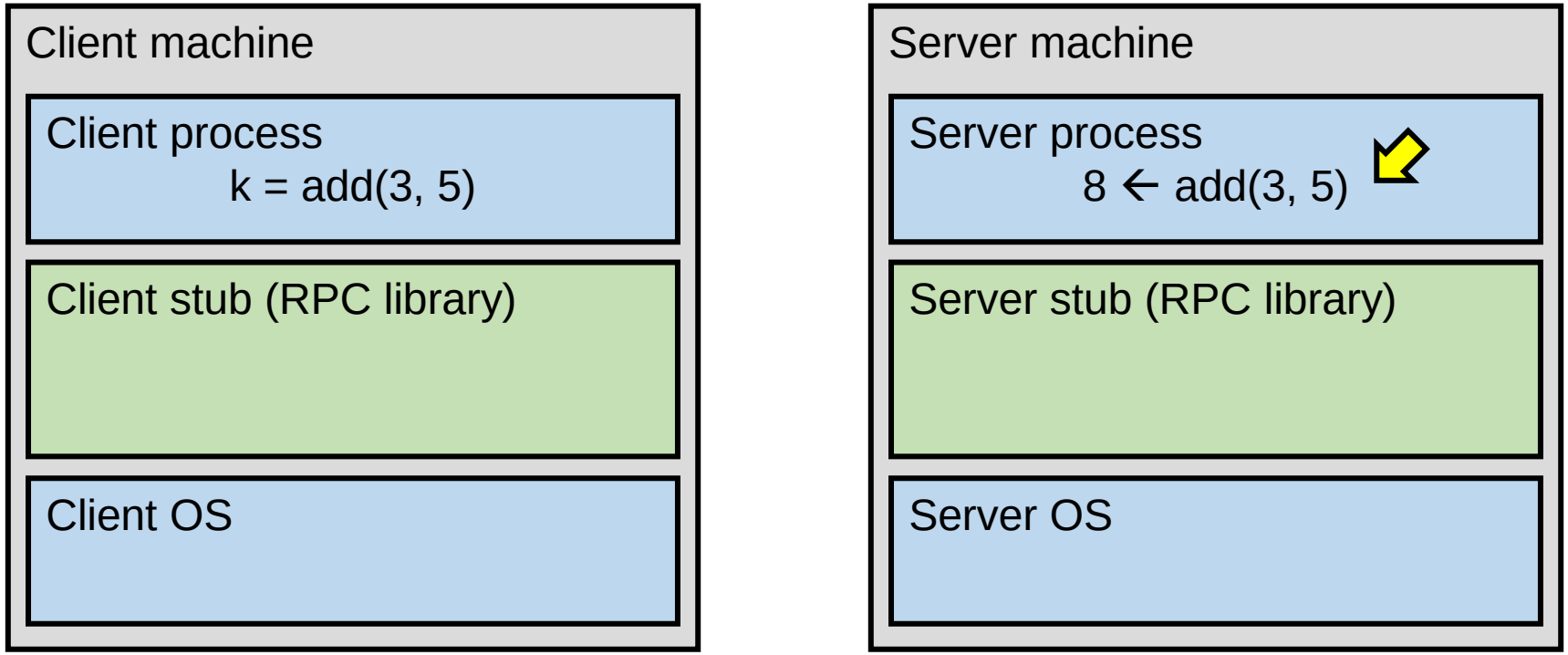
5. Server stub unmarshals params, calls server function



# A day in the life of an RPC

5. Server stub unmarshals params, calls server function

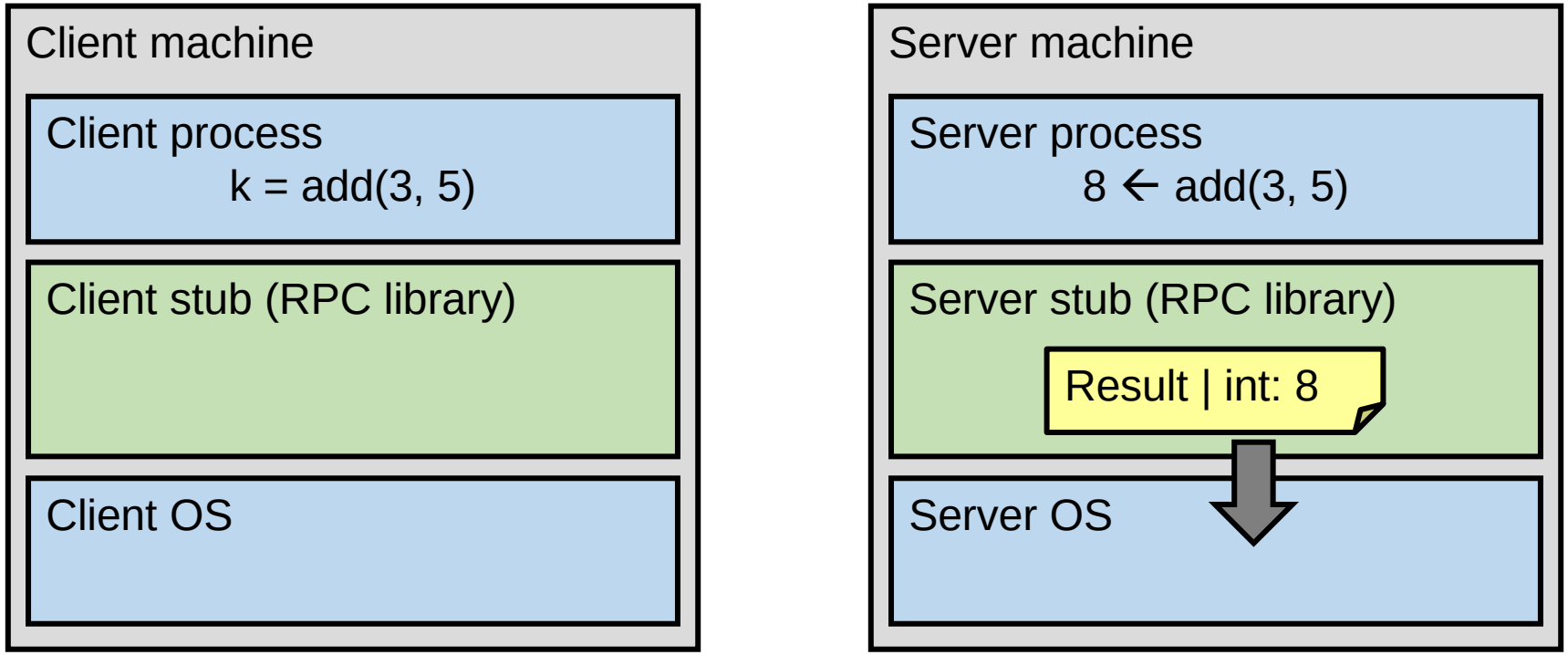
6. Server function runs, returns a value



# A day in the life of an RPC

6. Server function runs, returns a value

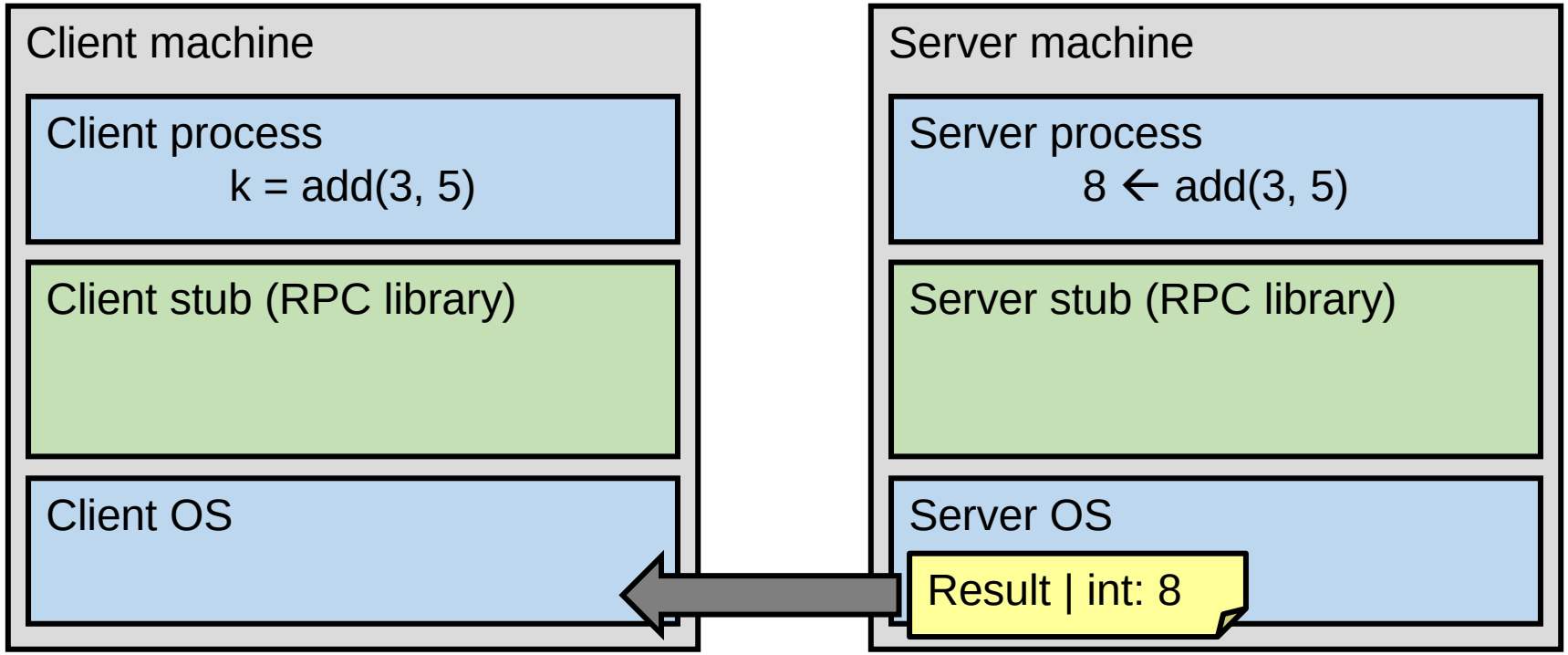
7. Server stub marshals the return value, sends message



# A day in the life of an RPC

7. Server stub marshals the return value, sends message

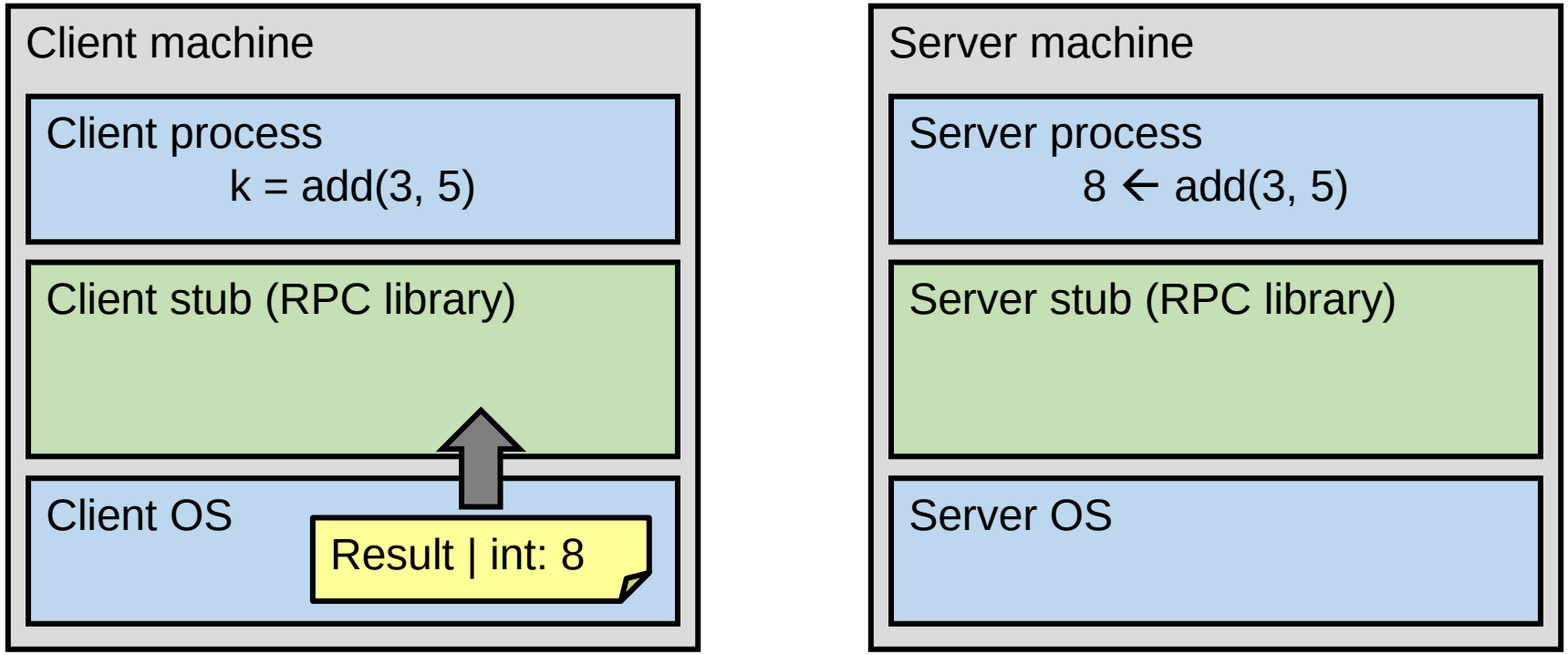
8. Server OS sends the reply back across the network



# A day in the life of an RPC

8. Server OS sends the reply back across the network

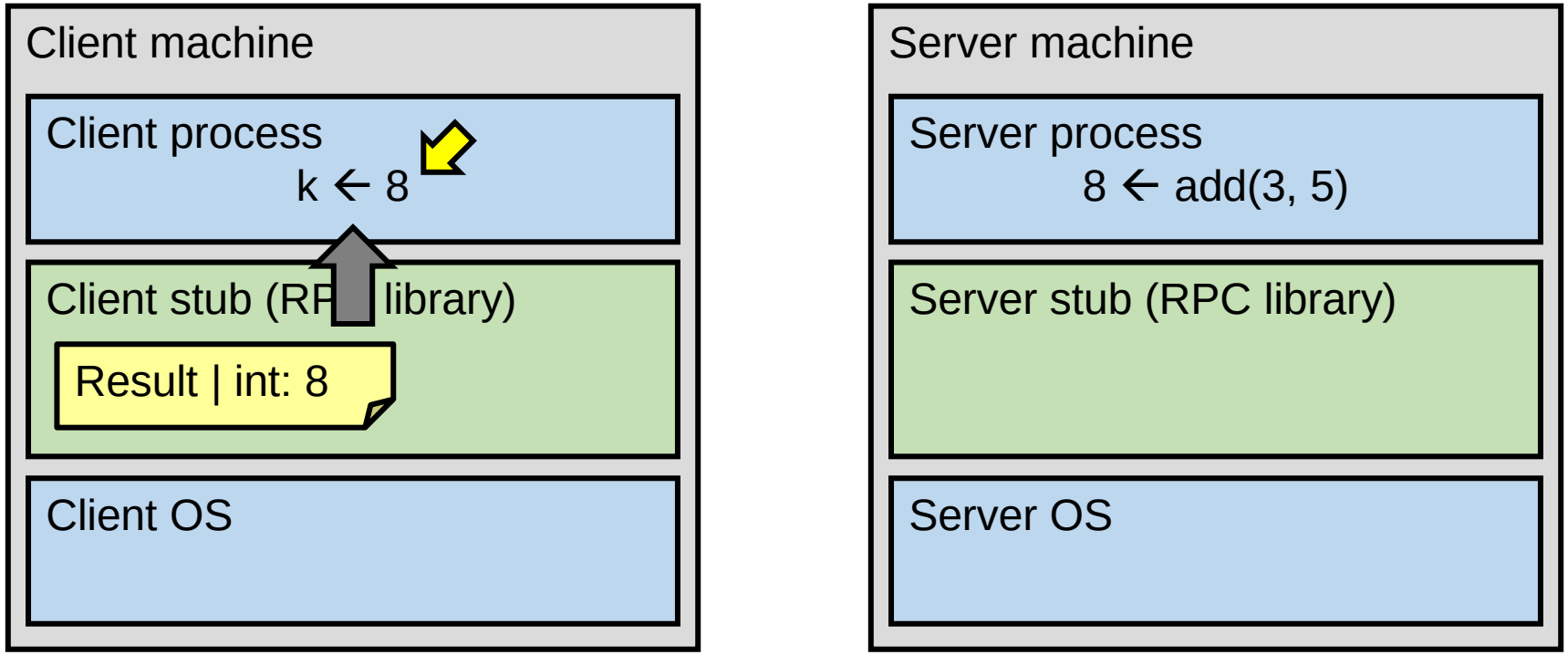
9. Client OS receives the reply and passes up to stub



# A day in the life of an RPC

9. Client OS receives the reply and passes up to stub

10. Client stub unmarshals return value, returns to client



# RPC: Parameter passing

There's more than just wrapping parameters into a message

- Client and server machines may have different data representations (think of byte ordering)
- Wrapping a parameter means transforming a value into a sequence of bytes
- Client and server have to agree on the same encoding:

- How are basic data values represented (integers, floats, characters)
- How are complex data values represented (arrays, unions)

## Conclusion

Client and server need to properly interpret messages, transforming them into machine-dependent representations.

More on this shortly

# RPC: Parameter passing

Some assumptions

- Copy in/copy out semantics
  - Can we pass pointers in the RPC calls?
    - Note: A pointer value is meaningful only within the address space of the process in which it is being used. Simply passing the pointer value (address) will be wrong.
    - Either disallow pointers or handle them properly in the background, e.g., by passing a copy of the data structure pointed to by the pointer. (Issues?)
- All data that is to be operated on is **passed by parameters.**
  - Non-local variables of a process cannot be accessed
  - This excludes passing references to global data that all distributed systems nodes have access to.

Conclusion: Full access transparency cannot be realized.

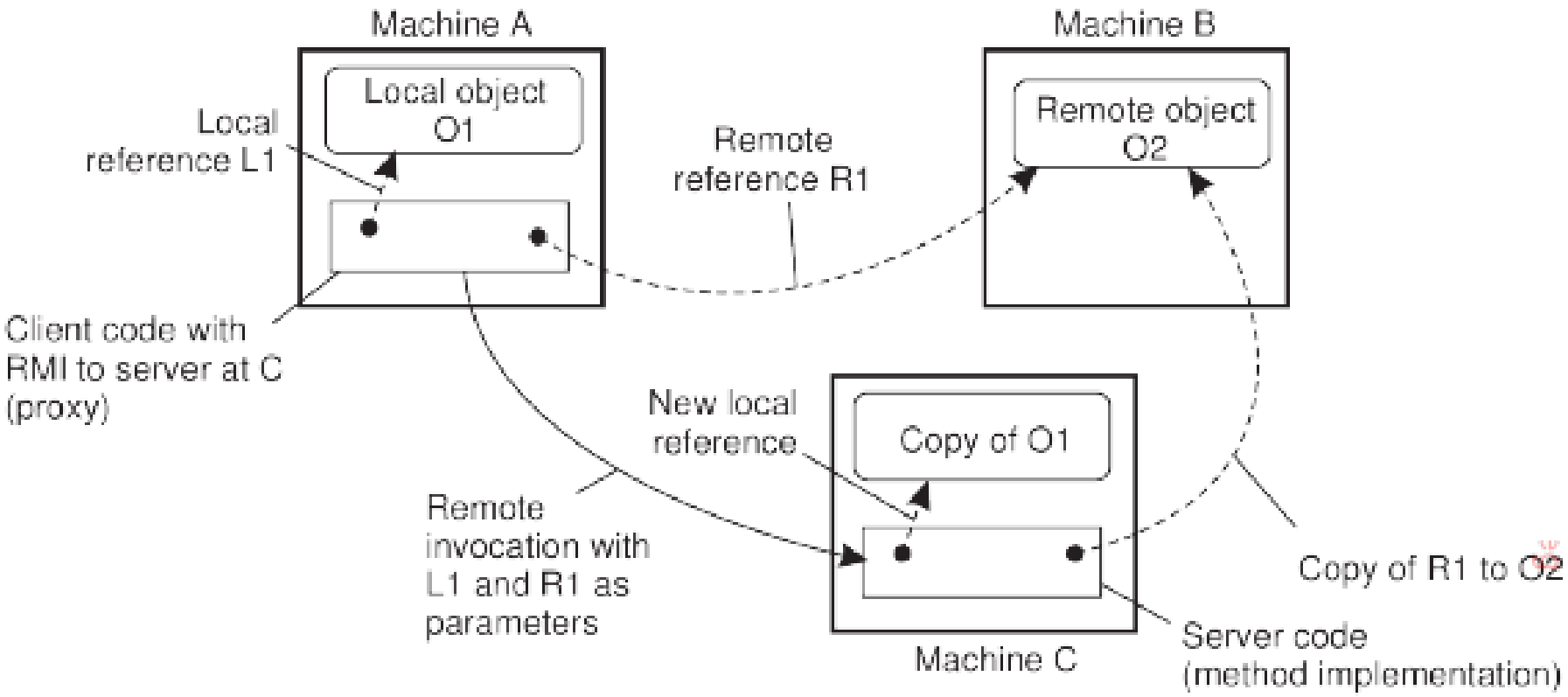
# RPC: Parameter passing

## Remote References

- Remote (global) references: references that are meaningful to the calling and the called process.
- A remote reference mechanism enhances access transparency
  - Remote reference offers unified access to remote data
  - Remote references can be passed as parameter in RPCs

Would it be possible to provide full transparency in this case?

# (Advanced: Parameter passing in object-based systems)




Discussion:
- There were different interactions with objects O1 and O2, why?
- Would it be a good idea to make this fully transparent?

# RPC issues

1. Heterogeneity
   - Client needs to rendezvous with the server and have to agree on the same encoding.
   - Server must dispatch to the required function
     - What if server is different type of machine?
   - How to write server and client stubs that handle these issues?

2. Failure
   - What if messages get dropped?
   - What if client, server, or network fails?

3. Performance
   - Procedure call takes ≈ 10 cycles ≈ 3 ns
   - RPC in a data center takes ≈ 10 μs ($10^3$× slower)
     - In the wide area, typically $10^6$× slower

# Problem: Differences in data representation

- Not an issue for local procedure calls

- For a remote procedure call, a remote machine may:
  - Run process written in a different language
  - Represent data types using different sizes
  - Use a different byte ordering (endianness)
  - Represent floating point numbers differently
  - Have different data alignment requirements
    - e.g., 4-byte type begins only on 4-byte memory boundary

# How to write client/server stubs?

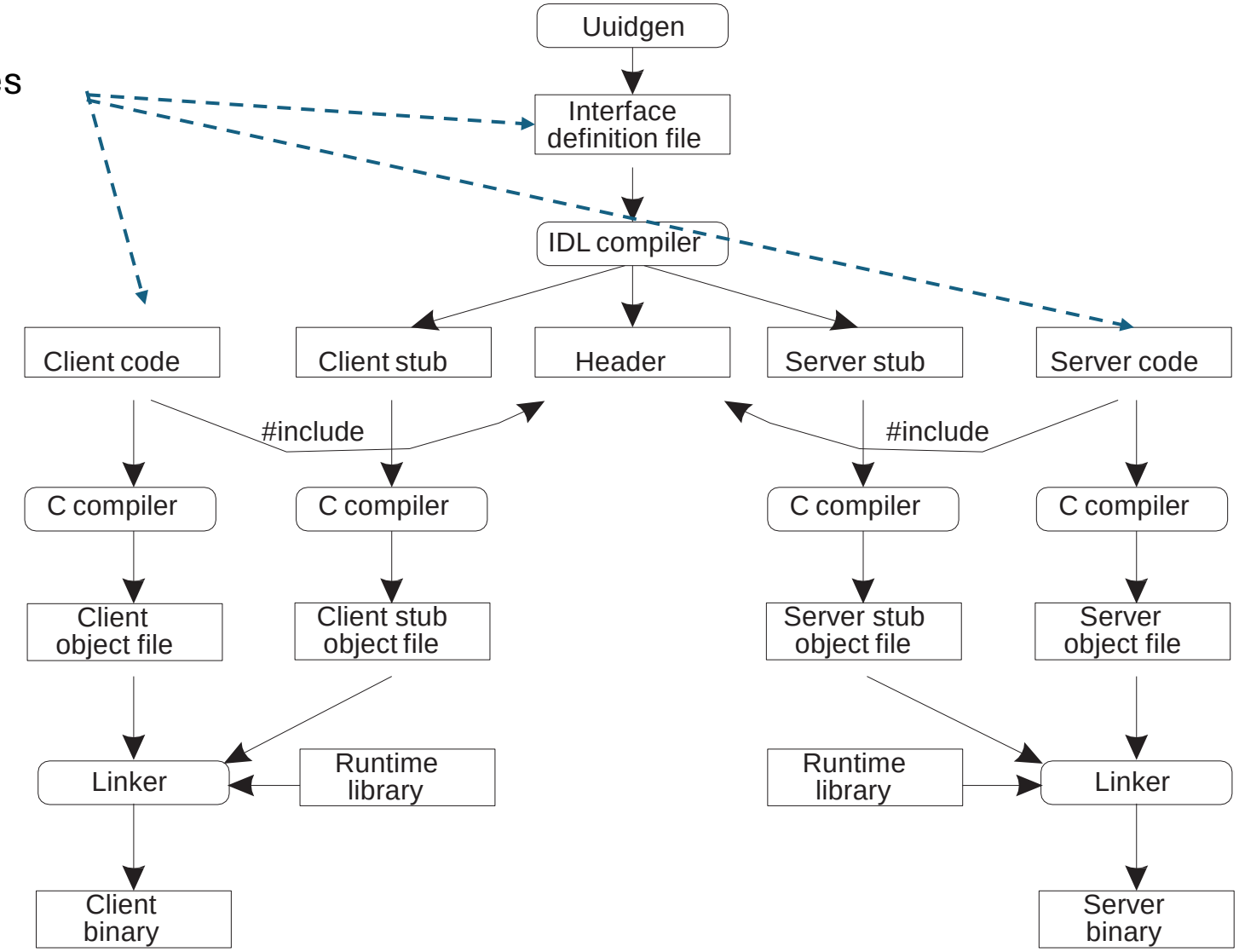
- Two approaches:
  - Approach 1:
    - Let the programmer write a description for the interface between the client and the server.
    - Use a tool to generate the code of the client and server stubs,
  - Approach 2:
    - Built-in support by the language

# Solution: Interface Description Language

- Mechanism to pass procedure parameters and return values in a machine-independent way
- Programmer may write an interface description in the IDL
  - Defines API for procedure calls: names, parameter/return types
- Then runs an IDL compiler which generates:
  - Code to marshal (convert) native data types into machine-independent byte streams
    - And vice-versa, called unmarshaling
  - Client stub: Forwards local procedure call as a request to server
  - Server stub: Dispatches RPC to its implementation

# RPC in practice

What the programmer writes



# Demo – RPC C Example

Based on the tutorial here: https://docs.oracle.com/cd/E19683-01/816-1435/rpcgenpguide-21470/index.html

IDL and generated code can be found here: https://github.com/docc-lab/tufts_comp_118_demos/tree/master/rpc_demo

See msg.x (This is the IDL file). The generated stubs are in the repo.

# IDL file example

```
struct messagelist {
   /* Char *s that represent null terminated strings need to be declared as
    * strings in XDR. This is because a char * is ambigous.  It could point to
    * a single character or to a NULL terminated string; XDR can't know
    * the difference.
    */
   string name<255>;  /* char* strings need to be declared as strings in XDR w/a max size */
   int count;
   messagelist *next;
};

program MESSAGEPROG {
     version PRINTMESSAGEVERS {
        /* The "1" below is a procedure number.  Every fn must have a unique numer. */
        int PRINTMESSAGE(struct messagelist *) = 1;
         } = 1;
} = 0x20000001;
```

# RPC in Go

# Go RPCs

Implementation in built-in library `net/rpc` requires 3 steps:

1. Write stub methods of the form

```
func (t *T) MethodName(args T1, reply *T2) error
```

2. Register RPC methods

3. Create a listener (i.e., server) that accepts requests

# Go example: Word count server

```go
type WordCountServer struct {
    addr string
}

type WordCountRequest struct {
    Input string
}

type WordCountReply struct {
    Counts map[string]int
}
```

```go
func (*WordCountServer) Compute(
        request WordCountRequest,
        reply *WordCountReply) error {
    counts := make(map[string]int)
    input := request.Input
    tokens := strings.Fields(input)
    for _, t := range tokens {
        counts[t] += 1
    }
    reply.Counts = counts
    return nil
}
```

# Go example: Word count server

```go
func (server *WordCountServer) Listen() {
    rpc.Register(server)
    listener, err := net.Listen("tcp", server.addr)
    checkError(err)
    go func() {
        rpc.Accept(listener)
    }()
}
```

# Go example: Word count client

```go
func makeRequest(input string, serverAddr string) (map[string]int, error) {
    client, err := rpc.Dial("tcp", serverAddr)
    checkError(err)
    args := WordCountRequest{input}
    reply := WordCountReply{make(map[string]int)}
    err = client.Call("WordCountServer.Compute", args, &reply)
    if err != nil {
        return nil, err
    }
    return reply.Counts, nil
}
```

# Go example: Word count client

```go
func makeRequest(input string, serverAddr string) (map[string]int, error) {
    client, err := rpc.Dial("tcp", serverAddr)
    checkError(err)
    args := WordCountRequest{input}
    reply := WordCountReply{make(map[string]int)}
    err = client.Call("WordCountServer.Compute", args, &reply)
    if err != nil {
        return nil, err
    }
    return reply.Counts, nil
}
```

# Go example: Word count client

```go
func makeRequest(input string, serverAddr string) (map[string]int, error) {
    client, err := rpc.Dial("tcp", serverAddr)
    checkError(err)
    args := WordCountRequest{input}
    reply := WordCountReply{make(map[string]int)}
    err = client.Call("WordCountServer.Compute", args, &reply)
    if err != nil {
        return nil, err
    }
    return reply.Counts, nil
}
```

# Go example: Word count client-server

```
func main() {
    serverAddr := "localhost:8888"
    server := WordCountServer{serverAddr}
    server.Listen()
    input1 := "hello I am good hello bye bye bye bye good night
hello"
    wordcount, err := makeRequest(input1, serverAddr)
    checkError(err)
```

```
Result: map[hello:3 I:1 am:1 good:2 bye:4
night:1]
```

# Is this synchronous or asynchronous?

```go
func makeRequest(input string, serverAddr string) (map[string]int, error) {
    client, err := rpc.Dial("tcp", serverAddr)
    checkError(err)
    args := WordCountRequest{input}
    reply := WordCountReply{make(map[string]int)}
    err = client.Call("WordCountServer.Compute", args, &reply)
    if err != nil {
        return nil, err
    }
    return reply.Counts, nil
}
```

# Making client asynchronous

```go
func makeRequest(input string, serverAddr string) chan Result {
    client, err := rpc.Dial("tcp", serverAddr)
    checkError(err)
    args := WordCountRequest{input}
    reply := WordCountReply{make(map[string]int)}
    ch := make(chan Result)
    go func() {
        err := client.Call("WordCountServer.Compute", args, &reply)
        if err != nil {
            ch <- Result{nil, err} // something went wrong
        } else {
            ch <- Result{reply.Counts, nil} // success
        }
    }()
    return ch
}
```

# Making client asynchronous

```go
func makeRequest(input string, serverAddr string) *Call {
    client, err := rpc.Dial("tcp", serverAddr)
    checkError(err)
    args := WordCountRequest{input}
    reply := WordCountReply{make(map[string]int)}
    return client.Go("WordCountServer.Compute", args, &reply, nil)
}

call := makeRequest(...)
<-call.Done
checkError(call.Error)
handleReply(call.Reply)
```

# More on RPCs in Go

- The previous showed the native RPC support using net/rpc lib.

- Another RPC variant is gRPC
  - The programmer writes an IDL in ProtoBuf, and stubs in go are generated accordingly by a compiler (source)
  - See examples in https://github.com/grpc/grpc-go/tree/master

```
syntax = "proto3";

option go_package = "google.golang.org/grpc/examples/helloworld/helloworld";
option java_multiple_files = true;
option java_package = "io.grpc.examples.helloworld";
option java_outer_classname = "HelloWorldProto";

package helloworld;

// The greeting service definition.
service Greeter {
  // Sends a greeting
  rpc SayHello (HelloRequest) returns (HelloReply) {}
}

// The request message containing the user's name.
message HelloRequest {
  string name = 1;
}

// The response message containing the greetings
message HelloReply {
  string message = 1;
}
```

# RPC: Fault Tolerance

# What could possibly go wrong?

1. Client may crash and reboot

2. Packets may be dropped
   - Some individual packet loss in the Internet
   - Broken routing results in many lost packets

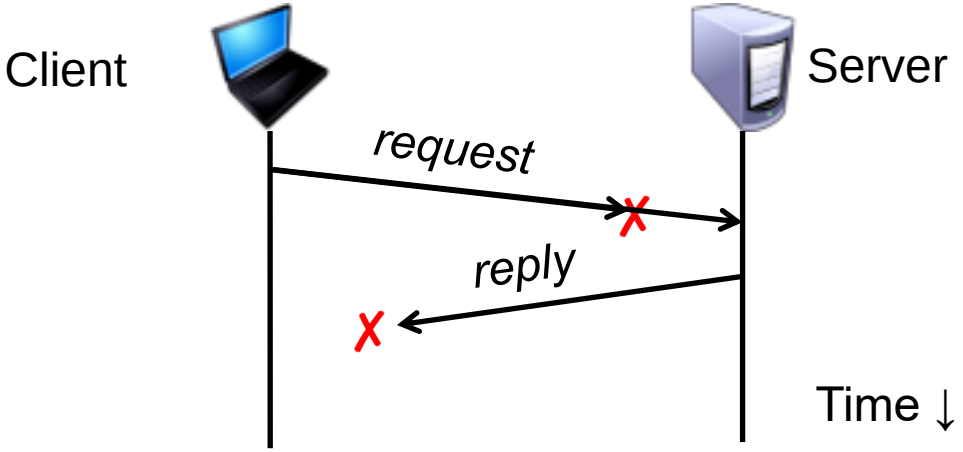
3. Server may crash and reboot

4. Network or server might just be very slow

All of these may look the same to the client…

- Note: The next discussion does not make any assumptions about the underlying transport layer protocol used in the communication (it could be TCP or UDP).

# Failures, from client's perspective




The cause of the failure is hidden from the client!

# At-Least-Once scheme

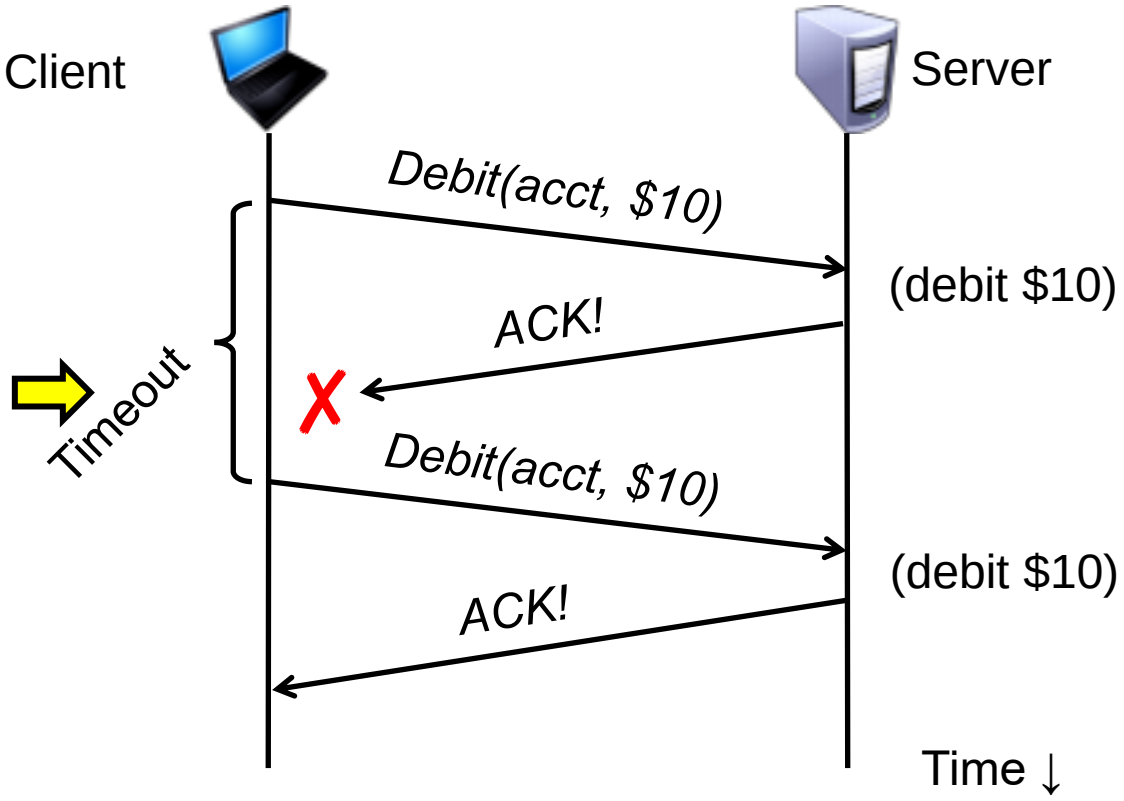
- Simplest scheme for handling failures

1. Client stub waits for a response, for a while
   - Response is an acknowledgement message from the server stub
2. If no response arrives after a fixed timeout time period, then client stub re-sends the request

- Repeat the above a few times
  - Still no response? Return an error to the application

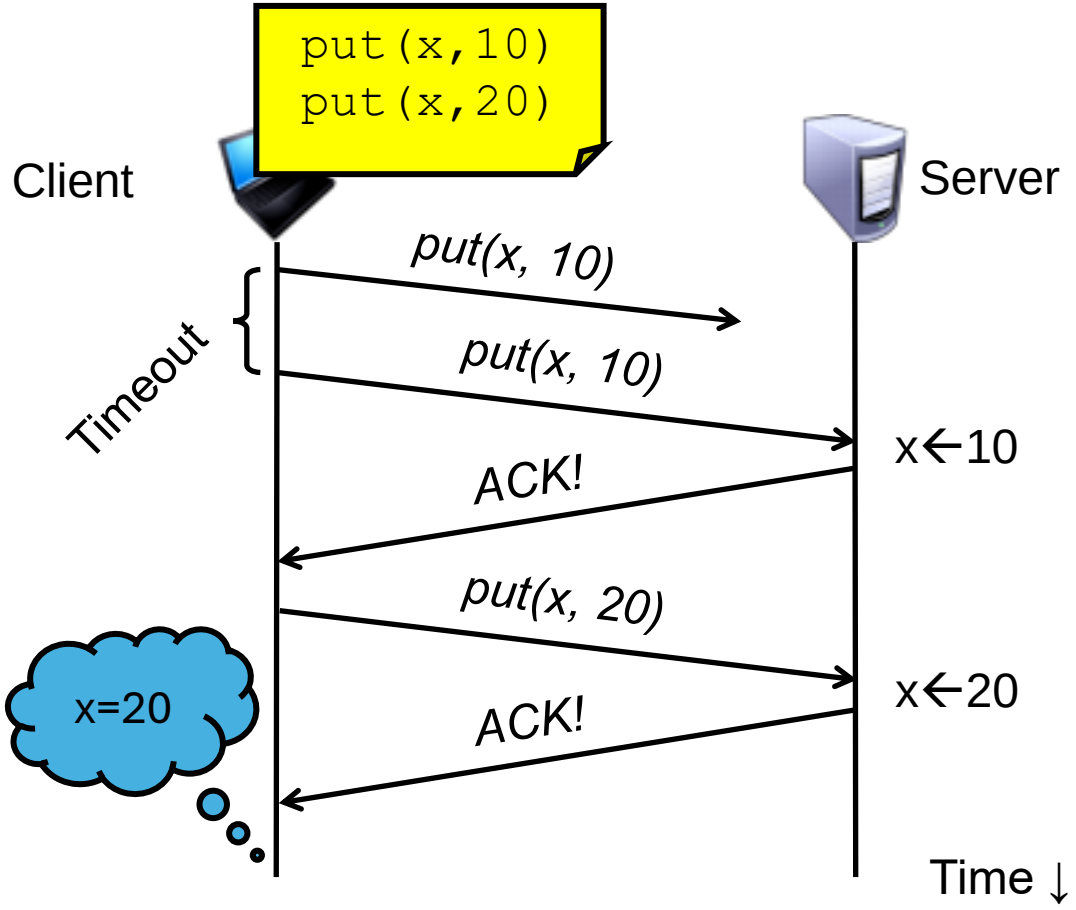
# At-Least-Once and side effects

- Client sends a “debit $10 from bank account” RPC



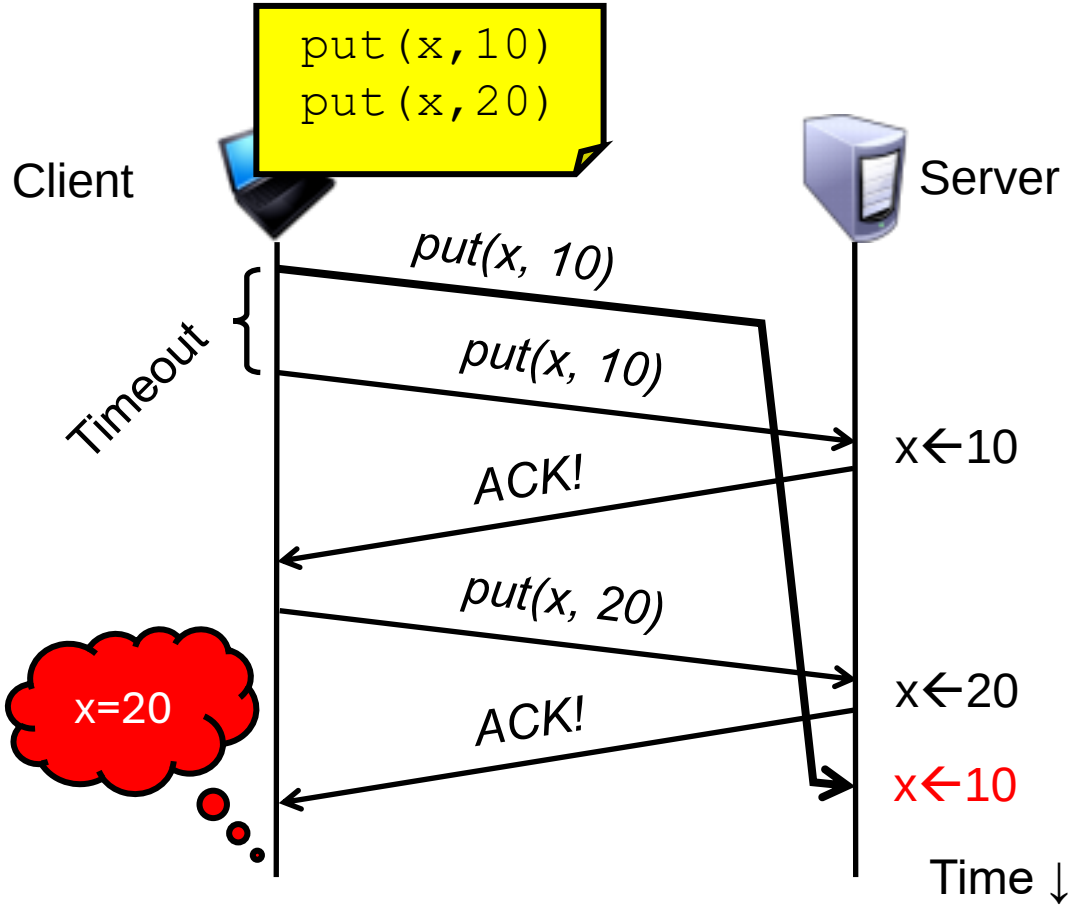
# At-Least-Once and writes

- put(x, *value*), then get(x): expect answer to be *value*



# At-Least-Once and writes

- Consider a client storing key-value pairs in a database
  - put(x, value), then get(x): expect answer to be value



# So is At-Least-Once ever okay?

- Yes: If they are read-only operations with no side effects
  - e.g., read a key's value in a database

- Yes: If the application has its own functionality to cope with duplication and reordering

# At-Most-Once scheme

- Idea: server RPC stub detects duplicate requests
  - Returns previous reply instead of re-running handler

- How to detect a duplicate request?
  - Test: Server stub sees same function, same arguments twice
    - No! Sometimes applications legitimately submit the same function with same augments, twice in a row

# At-Most-Once scheme

- How to detect a duplicate request?
  - Client stub includes unique transaction ID (xid) with each RPC request
  - Client stub uses same xid for retransmitted requests

```
At-Most-Once Server Stub
if seen[xid]:
        retval = old[xid]
else:
        retval = handler()
        old[xid] = retval
        seen[xid] = true
return retval
```

# At-Most-Once: Providing unique XIDs

1. Combine a unique client ID (e.g., IP address) with the current time of day

2. Combine unique client ID with a sequence number
   - Suppose client crashes and restarts. Can it reuse the same client ID?

3. Big random number (probabilistic, not certain guarantee)

# At-Most-Once: Discarding server state

- Problem: seen and old arrays will grow without bound

- Observation: By construction, when the client gets a response to a particular xid, it will never re-send it

- Client could tell server "I'm done with xid x – delete it"
  - Have to tell the server about each and every retired xid
    - Could piggyback on subsequent requests

Significant overhead if many RPCs are in flight, in parallel

# At-Most-Once: Discarding server state

- Problem: seen and old arrays will grow without bound
- Suppose xid = ⟨unique client id, sequence no.⟩
  - e.g., ⟨42, 1000⟩, ⟨42, 1001⟩, ⟨42, 1002⟩
- Client includes "seen all replies ≤ X" with every RPC
  - Much like TCP sequence numbers, acks
- How does the client know that the server received the information about retired RPCs?
  - Each one of these is cumulative: later seen messages subsume earlier ones

# At-Most-Once: Concurrent requests

- Problem: How to handle a duplicate request while the original is still executing?
  - Server doesn't know reply yet. Also, we don't want to run the procedure twice
- Idea: Add a `pending` flag per executing RPC
  - Server waits for the procedure to finish, or ignores

# At-Most-Once: Server crash and restart

- Problem: Server may crash and restart

- Does server need to write its tables to disk?

- Yes! On server crash and restart:
  - If `old[]`, `seen[]` tables are only in memory:
    - Server will forget, accept duplicate requests

# Exactly-once?

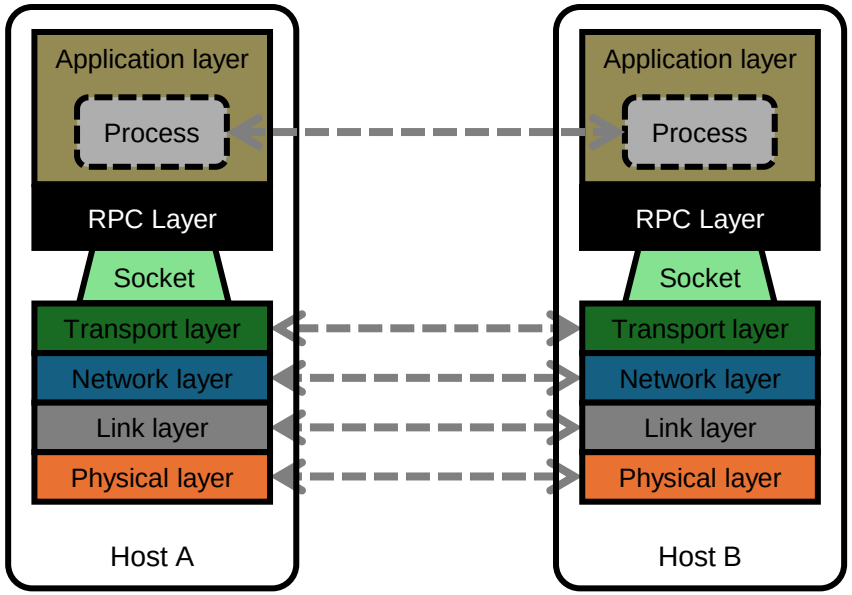
- Need retransmission of at least once scheme

- Plus the duplicate filtering of at most once scheme
  - To survive client crashes, client needs to record pending RPCs on disk
    - So it can replay them with the same unique identifier

- Plus story for making server reliable
  - Even if server fails, it needs to continue with full state
  - To survive server crashes, server should log to disk results of completed RPCs (to suppress duplicates)

# Exactly-once for external actions?

- Imagine that the remote operation triggers an external physical thing
  - e.g., dispense $100 from an ATM or printing on a screen
- The ATM could crash immediately before or after dispensing and lose its state
  - Don't know which one happened
    - Can, however, make this window very small
- So can't achieve exactly-once in general, in the presence of external actions

# Summary: RPCs and Network Comm.

- Layers are our friends!
- RPCs are everywhere
- Necessary issues surrounding machine heterogeneity
- Subtle issues around failures
  - At-least-once w/ retransmission
  - At-most-once w/ duplicate filtering
    - Discard server state w/ cumulative acks
  - Exactly-once with:
    - at-least-once + at-most-once + fault tolerance + no external actions